

▼ Sistema de recomendação

Baseado no exemplo de Kuldeep Singh disponível em:

(<https://github.com/singhsidhukuldeep/Recommendation-System>)

Como construir um sistema de recomendação usando scikit-surprise (biblioteca surprise)

Duas das formas mais populares de abordar os sistemas de recomendação são [filtragem colaborativa] e [recomendações baseadas em conteúdo].

Neste tutorial/exemplo, vamos nos concentrar na abordagem de **filtragem colaborativa**, ou seja: o usuário recomenda itens que pessoas com gostos e preferências semelhantes gostavam no passado. Em outras palavras, esse método prevê classificações desconhecidas usando as semelhanças entre os usuários.

▼ Instalando as bibliotecas necessárias

Surprise é um scikit Python para construir e analisar sistemas de recomendação.

<http://surpriselib.com/>

Surprise fornece vários algoritmos de previsão prontos para uso, como algoritmos de linha de base, métodos de vizinhança (k-nn), baseados em fatoração de matriz (SVD, PMF, SVD ++, NMF) e muitos outros. Além disso, várias medidas de similaridade (cosseno, MSD, pearson ...) estão embutidas. Facilita a implementação de novas idéias de algoritmos. Fornece ferramentas para avaliar, analisar e comparar o desempenho dos algoritmos. Os procedimentos de validação cruzada podem ser executados facilmente usando poderosos iteradores de CV (inspirados nas excelentes ferramentas do scikit-learn), bem como uma pesquisa exaustiva sobre um conjunto de parâmetros. O nome SurPRISE significa Simple Python Recommendation System Engine.

Observe que a surpresa não oferece suporte a classificações implícitas ou informações baseadas em conteúdo.

```
!pip3 install numpy
!pip3 install scikit-surprise
```

▼ Importando bibliotecas necessárias

```
import pandas as pd
from surprise import Reader
from surprise import Dataset
from surprise.model_selection import cross_validate
#preditores da biblioteca
from surprise import NormalPredictor
from surprise import KNNBasic
from surprise import ...
```

```
from surprise import KNNWithMeans
from surprise import KNNWithZScore
from surprise import KNNBaseline
from surprise import SVD
from surprise import BaselineOnly
from surprise import SVDpp
from surprise import NMF
from surprise import SlopeOne
from surprise import CoClustering
# metricas de avaliacao
from surprise.accuracy import rmse
from surprise import accuracy
from surprise.model_selection import train_test_split
from surprise.model_selection import GridSearchCV
```

▼ Coletando os dados

O GroupLens Research coletou e disponibilizou conjuntos de dados de classificação do site MovieLens ([\[http://movielens.org\]](http://movielens.org) (<http://movielens.org/>)). Os conjuntos de dados foram coletados em vários períodos de tempo, dependendo do tamanho do conjunto.

Estamos usando um arquivo *Pequeno*: 100.000 classificações e 3.600 aplicativos de tag aplicados a 9.000 filmes por 600 usuários. Última atualização em 9/2018.

Download: [ml-latest-small.zip] (<http://files.grouplens.org/datasets/movielens/ml-latest-small.zip>) (tamanho: 1 MB)

```
df = pd.read_csv ("ratings.csv")
```

O arquivo ratings possui os campos: userId

- userId (usuário que fez a avaliação)
- movieId (filme avaliado)
- rating (avaliação)
- timestamp (quando foi feita a avaliação)

```
df.head()
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

```
# não precisamos a marca do tempo
df.drop(['timestamp'], axis=1, inplace=True)
df.columns = ['userID', 'item', 'rating']
```

```
df.head()
```

	userID	item	rating
0	1	1	4.0
1	1	3	4.0
2	1	6	4.0
3	1	47	5.0
4	1	50	5.0

```
print("Quantidade de avaliações: ", df.shape[0])
```

```
Quantidade de avaliações 100836
```

▼ Limpeza da base: Cold Start

Para evitar problemas de "cold start" eliminaremos da nossa base os filmes que quase nunca foram avaliados

```
# filtramos os itens que tiveram mais do que 5 avaliações
min_ratings = 5
filter_items = df['item'].value_counts() > min_ratings
filter_items = filter_items[filter_items.index.tolist()]

# filtramos os itens que tiveram mais do que 5 usuários avaliando
min_user_ratings = 5
filter_users = df['userID'].value_counts() > min_user_ratings
filter_users = filter_users[filter_users.index.tolist()]

# removemos da base aqueles que não atendem a essas seleções
df_new = df[(df['item'].isin(filter_items)) & (df['userID'].isin(filter_users))]

print('Quantidade original de dados:\t{}'.format(df.shape[0]))
print('Quantidade após o filtro:\t{}'.format(df_new.shape[0]))

Quantidade original de dados:    100836
Quantidade após o filtro:       88364
```

▼ Surprise

Para carregar um conjunto de dados de um dataframe do pandas, usaremos o método `load_from_df()`, também precisaremos de um objeto Reader e o parâmetro `rating_scale` deve

ser especificado. O dataframe deve ter três colunas, correspondendo aos ids do usuário, os ids dos itens e as classificações nesta ordem. Cada linha corresponde, portanto, a uma determinada classificação.

```
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df_new[['userID', 'item', 'rating']], reader)
```

▼ Treinando e Otimizando os modelos

NormalPredictor

O algoritmo NormalPredictor prevê uma classificação aleatória com base na distribuição do conjunto de treinamento, que é considerado normal. Este é um dos algoritmos mais básicos que não funcionam muito.

BaselineOnly

O algoritmo BasiclineOnly prevê a estimativa da linha de base para determinado usuário e item.

Algoritmos k-NN

KNNBasic é um algoritmo básico de filtragem colaborativa.

KNNWithMeans é um algoritmo básico de filtragem colaborativa, levando em consideração as avaliações médias de cada usuário.

KNNWithZScore é um algoritmo básico de filtragem colaborativa, levando em consideração a normalização do z-score de cada usuário.

KNNBaseline é um algoritmo básico de filtragem colaborativa que leva em consideração uma classificação de linha de base.

Algoritmos baseados em fatoração de matriz

SVD O algoritmo SVD é equivalente a Probabilistic Matrix Factorization (<http://papers.nips.cc/paper/3208-probabilistic-matrix-factorization.pdf>)

SVDpp O algoritmo SVDpp é uma extensão do SVD que leva em consideração as classificações implícitas.

NMF é um algoritmo de filtragem colaborativa baseado na Fatoração de Matrizes Não Negativas. É muito semelhante ao SVD.

Slope One Slope One é uma implementação direta do algoritmo SlopeOne. (<https://arxiv.org/abs/cs/0702144>)

Co-clustering Co-clustering é um algoritmo de filtragem colaborativa baseado em co-clustering (<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.113.6458&rep=rep1&type=pdf>)

Usamos RMSE como nossa métrica de precisão para as previsões.

```
benchmark = []

# algoritmos que serão testados
algorithms = [SVD(), SVDpp(), SlopeOne(), NMF(), NormalPredictor(), KNNBaseline(), KNNBasi

# treinando e avaliando cada algoritmo
for algorithm in algorithms:
    print("Iniciando: ", str(algorithm.__class__.__name__))
    # validação cruzada
    results = cross_validate(algorithm, data, measures=['RMSE'], cv=3, verbose=False)
    # coleta dos resultados
    tmp = pd.DataFrame.from_dict(results).mean(axis=0)
    tmp = tmp.append(pd.Series([str(algorithm).split(' ')[0].split('.')[1]], index=['Algo
benchmark.append(tmp)

print ('\n\tPronto\n')
```

```
Iniciando:  SVD
Iniciando:  SVDpp
Iniciando:  SlopeOne
Iniciando:  NMF
Iniciando:  NormalPredictor
Iniciando:  KNNBaseline
Estimating biases using als...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using als...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using als...
Computing the msd similarity matrix...
Done computing similarity matrix.
Iniciando:  KNNBasic
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Iniciando:  KNNWithMeans
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Iniciando:  KNNWithZScore
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Iniciando:  BaselineOnly
Estimating biases using als...
Estimating biases using als...
Estimating biases using als...
```

Iniciando: CoClustering

DONE

```
# construímos um DataFrame ordenado pelo valor do RMSE
surprise_results = pd.DataFrame(benchmark).set_index('Algorithm').sort_values('test_rmse')
```

surprise_results

	test_rmse	fit_time	test_time
Algorithm			
SVDpp	0.849428	220.436080	11.849936
KNNBaseline	0.854375	0.403863	3.104876
BaselineOnly	0.860623	0.323175	0.229772
SVD	0.862735	3.563683	0.336143
KNNWithZScore	0.866158	0.183045	2.764661
KNNWithMeans	0.867064	0.132641	2.542381
SlopeOne	0.871751	1.439501	8.652410
NMF	0.897752	4.539199	0.250918
CoClustering	0.919190	2.059778	0.247393
KNNBasic	0.926754	0.102578	2.351369
NormalPredictor	1.402098	0.128883	0.275105

▼ Otimização de Parâmetros

Escolhemos o SVD ou outro que possua um bom resultado de RMSE. Então realizamos um GridSearchCV para otimizar os parâmetros.

```
# param_grid = {
#     "n_epochs": [5, 10, 15, 20, 30, 40, 50, 100],
#     "lr_all": [0.001, 0.002, 0.005],
#     "reg_all": [0.02, 0.08, 0.4, 0.6]
# }

# smaller grid for testing
param_grid = {
    "n_epochs": [10, 20],
    "lr_all": [0.002, 0.005],
    "reg_all": [0.02]
}
gs = GridSearchCV(SVD, param_grid, measures=["rmse", "mae"], refit=True, cv=5)

gs.fit(data)
```

```

training_parameters = gs.best_params["rmse"]

print("Melhor RMSE: \t", gs.best_score["rmse"])
print("Melhor MAE: \t", gs.best_score["mae"])
print("Melhor parâmetros: \t", gs.best_params["rmse"])

```

▼ Criando o modelo

Utilizando os melhores parâmetros obtidos criamos nosso modelo

```

from datetime import datetime
print(training_parameters)
reader = Reader(rating_scale=(1, 5))

print("\n\n\t\t Início\n\n")
start = datetime.now()

print("> Carregando dados...")
data = Dataset.load_from_df(df_new[['userID', 'item', 'rating']], reader)
print("> OK")

print("> Criando base de treinamento...")
trainset = data.build_full_trainset()
print("> OK")

print("> treinando o modelo...")
algo = SVD(n_epochs = training_parameters['n_epochs'], lr_all = training_parameters['lr_all'])
algo.fit(trainset)

end = datetime.now()
print(">> Finalizado em", (end-start).seconds, "segundos" )

# predicting
from pprint import pprint as pp

def itemRating(user, item):
    uid = str(user)
    iid = str(item)
    prediction = algo.predict(user, item, verbose=True)
    rating = prediction.est
    details = prediction.details
    uid = prediction.uid
    iid = prediction.iid
    true = prediction.r_ui
    ret = {
        'user': user,
        'item': item,
        'rating': rating,
        'details': details,
        'uid': uid,
        'iid': iid,
        'true': true
    }

```

```
        true : true
    }
    return ret
```

```
#prevendo qual a avaliação que um usuário daria para um item
print("Avaliação: ", itemRating(user = "610", item = "10"))
```

```
user: 610      item: 10      r_ui = None    est = 3.54    {'was_impossible': False
Avaliação: {'user': '610', 'item': '10', 'rating': 3.543813091304151, 'details': {'v
```

